

Potenzmengenkonstruktion und Abschlusseigenschaften regulärer Sprachen

(Finite Automata, Subset Construction, and Closure Properties)

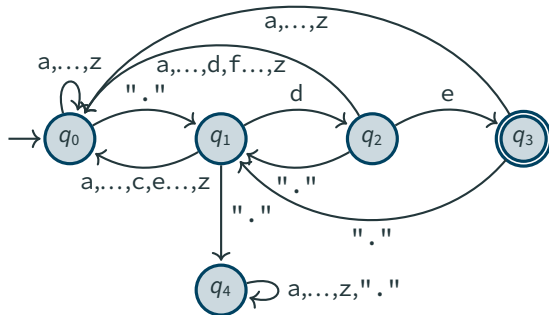
Theoretische Informatik

TH Lübeck

Nichtdeterministische endliche Automaten

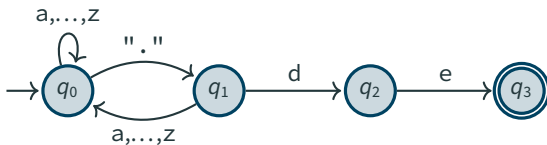
.de-DOMAINS

- Wort muss auf .de enden und keine zwei Punkte in Folge enthalten.
- Problem: .de kann auch mitten im Namen vorkommen (www.debian.org).
- DEA ist aufwändig.



.de-DOMAINS

- Wort muss auf .de enden und keine zwei Punkte in Folge enthalten.
- Problem: .de kann auch mitten im Namen vorkommen (www.**.de**bian.org).
- Einfache Lösung: Wir raten, wenn die letzten drei Buchstaben kommen.



- Wenn es *deterministische* endliche Automaten gibt...

- Wenn es *deterministische* endliche Automaten gibt...
- Dann auch *nichtdeterministische* Automaten
 - können in einem Zustand für das gleiche Zeichen *keinen, einen oder mehrere* Nachfolgezustände haben
 - suchen sich dann einen aus – raten –
 - akzeptieren, wenn der Automat beim Abarbeiten des Wortes in einem Endzustand enden kann – raten immer richtig –
 - können mehrere Anfangszustände haben und suchen sich einen aus – raten – in dem sie anfangen

Definition

Ein *nichtdeterministischer endlicher Automat* (engl. nondeterministic finite automaton), kurz *NEA* (engl. NFA), ist eine Struktur $A = (Q, \Sigma, \delta, Q_0, F)$ mit folgenden Eigenschaften:

- Q ist eine endliche Menge von **Zuständen**
- Σ ist eine endliche Menge, das **Eingabealphabet**
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ ist die **Überföhrungsfunktion**, die für jeden Zustand und Buchstaben eine Menge von Nachfolgezuständen liefert.
- $Q_0 \subseteq Q$ ist eine nicht-leere Menge von **Anfangszuständen**
- $F \subseteq Q$ ist die Menge der **Endzustände**.

Definition

Ein *nichtdeterministischer endlicher Automat* (engl. nondeterministic finite automaton), kurz *NEA* (engl. NFA), ist eine Struktur $A = (Q, \Sigma, \delta, Q_0, F)$ mit folgenden Eigenschaften:

- Q ist eine endliche Menge von **Zuständen**
- Σ ist eine endliche Menge, das **Eingabealphabet**
- $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ ist die **Überföhrungsfunktion**, die für jeden Zustand und Buchstaben eine Menge von Nachfolgezuständen liefert.
- $Q_0 \subseteq Q$ ist eine nicht-leere Menge von **Anfangszuständen**
- $F \subseteq Q$ ist die Menge der **Endzustände**.

$$\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$$

- bildet den aktuellen Zustand ($Q \times \Sigma \rightarrow \mathcal{P}(Q)$)
- und das aktuelle Eingabezeichen ($Q \times \Sigma \rightarrow \mathcal{P}(Q)$)
- auf eine Menge von Nachfolgezuständen ab. ($Q \times \Sigma \rightarrow \mathcal{P}(Q)$).

Wir schreiben kurz $q \xrightarrow{a} q'$ falls $q' \in \delta(q, a)$.

Definition (Lauf (Run) DEA)

Ein *Lauf* eines **DEA** $A = (Q, \Sigma, \delta, q_0, F)$ für ein Eingabewort $w = a_1 \dots a_n$ ist eine Folge $q_0, q_1, \dots, q_n$ von Zuständen, wobei

- q_0 der Startzustand von A ist und
- $\delta(q_{i-1}, a_i) = q_i$, d.h. A kommt von q_{i-1} mit Buchstabe a_i nach q_i .

Wir schreiben dann kurz: $q_0 \xrightarrow{w} q_n$.

Bei DEAs gibt es für jedes Wort **genau einen** Lauf.

Definition (Lauf (Run) NEA)

Ein *Lauf* eines **NEA** $A = (Q, \Sigma, \delta, Q_0, F)$ für ein Eingabewort $w = a_1 \dots a_n$ ist eine Folge $q_0, q_1, \dots, q_n$ von Zuständen, wobei

- $q_0 \in Q_0$ ein Startzustand von A ist und
- $\delta(q_{i-1}, a_i) \ni q_i$, d.h. A kommt von q_{i-1} mit Buchstabe a_i nach q_i .

Wir schreiben dann kurz: $q_0 \xrightarrow{w} q_n$.

Bei NEAs gibt es für jedes Wort **Null oder mehr** Läufe.

Definition (Akzeptanz DEA)

Ein **DEA** $A = (Q, \Sigma, \delta, q_0, F)$ akzeptiert ein Eingabewort $w = a_1 \dots a_n$ falls es einen Lauf $q_0, q_1, \dots, q_n$ für w gibt, der in einem Endzustand endet, d.h. $q_n \in F$.

Die akzeptierte Sprache ist die Menge der akzeptierten Wörter:

$$L(A) = \{w \in \Sigma^* \mid q_0 \xrightarrow{w} q_f \text{ and } q_f \in F\}.$$

Definition (Akzeptanz NEA)

Ein **NEA** $A = (Q, \Sigma, \delta, Q_0, F)$ akzeptiert ein Eingabewort $w = a_1 \dots a_n$ falls es einen Lauf $q_0, q_1, \dots, q_n$ für w gibt, der in einem Endzustand endet, d.h. $q_n \in F$.

Die akzeptierte Sprache ist die Menge der akzeptierten Wörter:

$$L(A) = \{w \in \Sigma^* \mid q_0 \xrightarrow{w} q_f \text{ and } q_f \in F\}.$$

1. Für NEAs liefert δ eine Menge von Nachfolgezuständen für einen Buchstaben. Damit sind auch Null Nachfolger möglich. Für DEAs liefert δ genau einen Nachfolger.
2. NEAs haben mindestens einen Startzustand, DEAs genau einen.
3. NEAs können keinen, einen oder mehr als einen Lauf für ein Wort haben. DEAs haben genau einen.
4. NEAs und DEAs akzeptieren, wenn es einen Lauf gibt, der in einen Endzustand führt.

DEA vs NEA

Jeder DEA ist also auch immer ein NEA, wenn man einelementige Mengen von Zuständen und Zustände nicht unterscheidet.

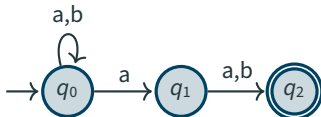
Beispiel

Betrachten Sie $L = \{waw' \in \Sigma^* \mid w \in \Sigma^*, w' \in \Sigma\}$ über dem Alphabet $\Sigma = \{a, b\}$.

Konstruieren Sie einen nichtdeterministischen endlichen Automaten für L .

Potenzmengenkonstruktion

- Erkenntnis 1: NEAs erlauben einfache Spezifikationen aber können nicht direkt in deterministischen Code umgesetzt werden.
- Erkenntnis 2: Jeder DEA ist ein NEA (bis auf formale Kleinigkeiten)
- Herausforderung: NEAs deterministisch machen
- Gute Nachricht: Ist möglich aber Zustandsmenge kann exponentiell wachsen



```
int checkStr(char* s) {
    int state = 0;
    char* rem = s;
    while (*rem) {
        switch (state) {
            case 0:
                if (*rem == 'a')
                    state = 1;
                else
                    state = 0;
                break;
            case 1:
                if (*rem == 'a')
                    state = 3;
                else
                    state = 2;
                break;
            case 2:
                if (*rem == 'a')
                    state = 1;
                else
                    state = 0;
                break;
            case 3:
                if (*rem == 'a')
                    state = 3;
                else
                    state = 2;
                break;
        }
        rem++;
    }
    return state == 2 || state == 3;
}
```

- Erkenntnis 1: NEAs erlauben einfache Spezifikationen aber können nicht direkt in deterministischen Code umgesetzt werden.
- Erkenntnis 2: Jeder DEA ist ein NEA (bis auf formale Kleinigkeiten)
- Herausforderung: NEAs deterministisch machen
- Gute Nachricht: Ist möglich aber Zustandsmenge kann exponentiell wachsen
- Potenzmengenkonstruktion nach Rabin und Scott



Michael Oser Rabin (Turing Award 1976)

Photo: Mathematisches Forschungsinstitut Oberwolfach,

http://owpdb.mfo.de/detail?photo_id=3420

Konrad Jacobs, Erlangen, Copyright MFO

Lizenz: Creative Commons Attribution-Share Alike 2.0 Germany

- Erkenntnis 1: NEAs erlauben einfache Spezifikationen aber können nicht direkt in deterministischen Code umgesetzt werden.
- Erkenntnis 2: Jeder DEA ist ein NEA (bis auf formale Kleinigkeiten)
- Herausforderung: NEAs deterministisch machen
- Gute Nachricht: Ist möglich aber Zustandsmenge kann exponentiell wachsen
- Potenzmengenkonstruktion nach Rabin und Scott



Dana Steward Scott (Turing Award 1976)

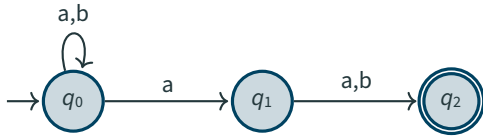
Photo: Andrej Bauer

<http://andrej.com/mathematicians>

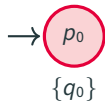
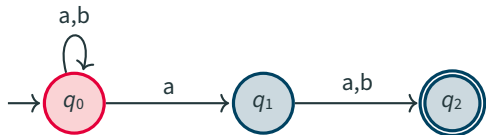
Lizenz: Creative Commons Attribution-Share Alike 2.5 Slovenia

- Starte mit NEA
- Baue DEA;
 - Merkt sich im Zustand in welchen Zuständen der NEA gerade sein könnte
 - Nutzt dazu die Teilmengen der Zustandsmenge des NEA als Zustände

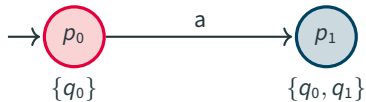
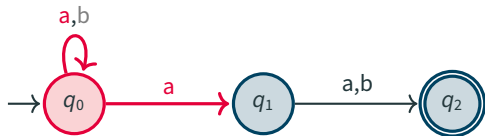
POTENZMENGENKONSTRUKTION – BEISPIEL



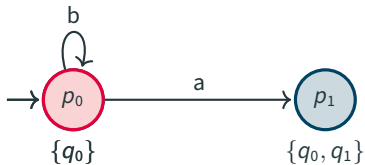
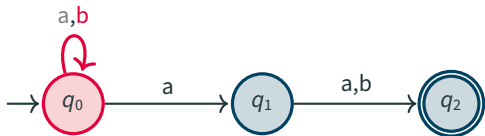
POTENZMENGENKONSTRUKTION – BEISPIEL



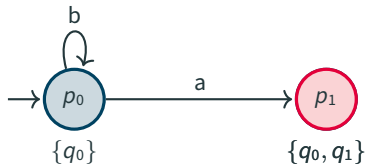
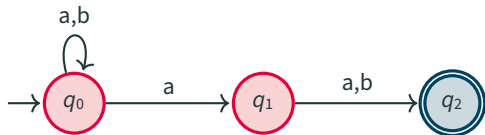
POTENZMENGENKONSTRUKTION – BEISPIEL



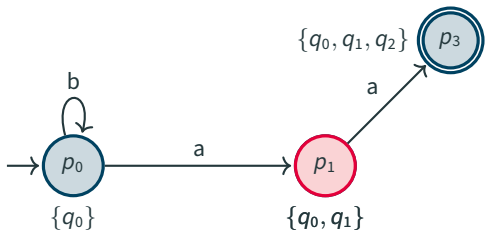
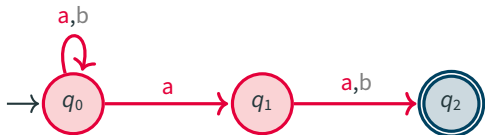
POTENZMENGENKONSTRUKTION – BEISPIEL



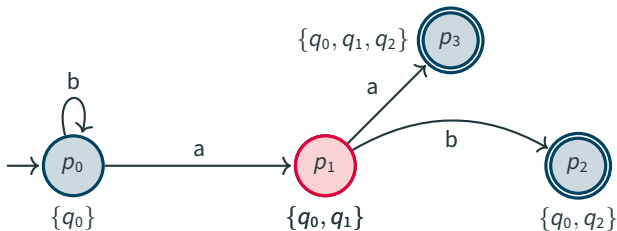
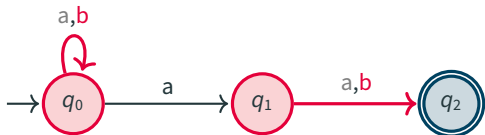
POTENZMENGENKONSTRUKTION – BEISPIEL



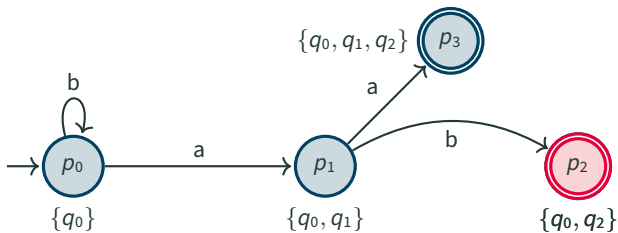
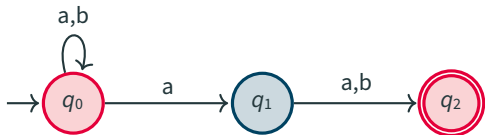
POTENZMENGENKONSTRUKTION – BEISPIEL



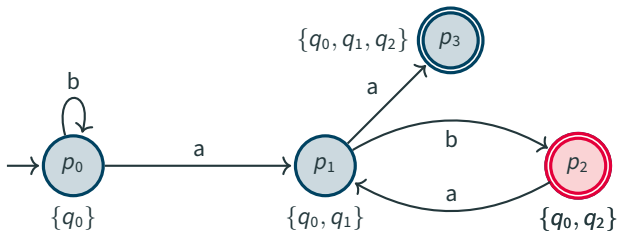
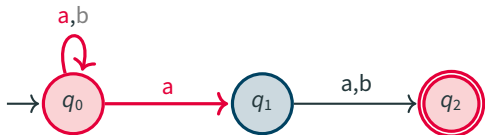
POTENZMENGENKONSTRUKTION – BEISPIEL



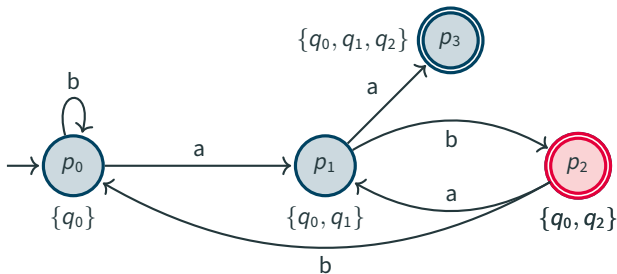
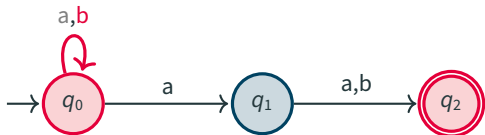
POTENZMENGENKONSTRUKTION – BEISPIEL



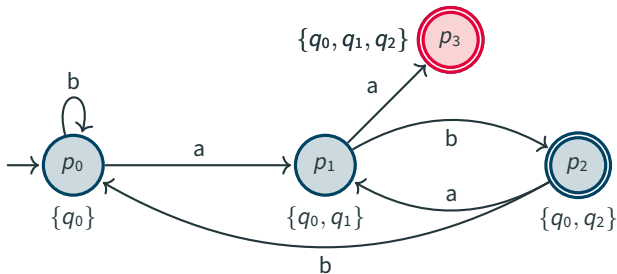
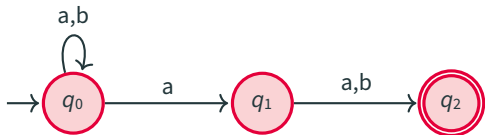
POTENZMENGENKONSTRUKTION – BEISPIEL



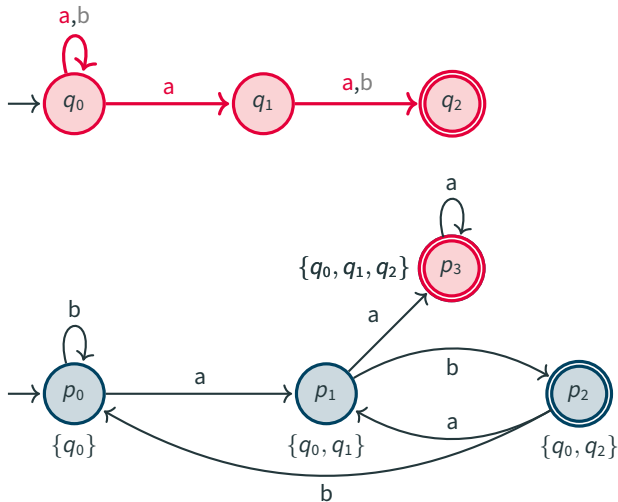
POTENZMENGENKONSTRUKTION – BEISPIEL



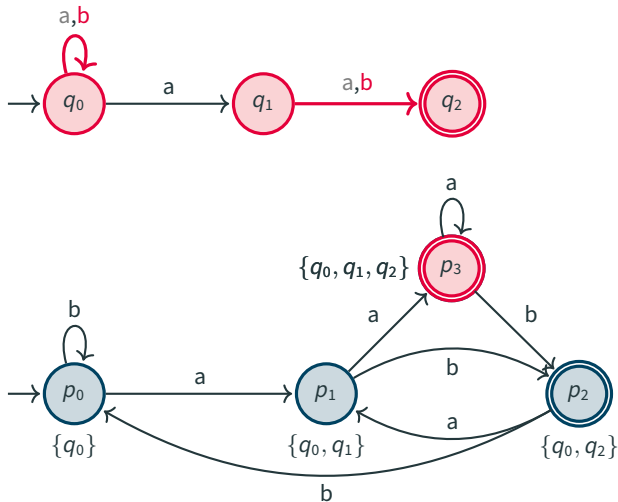
POTENZMENGENKONSTRUKTION – BEISPIEL



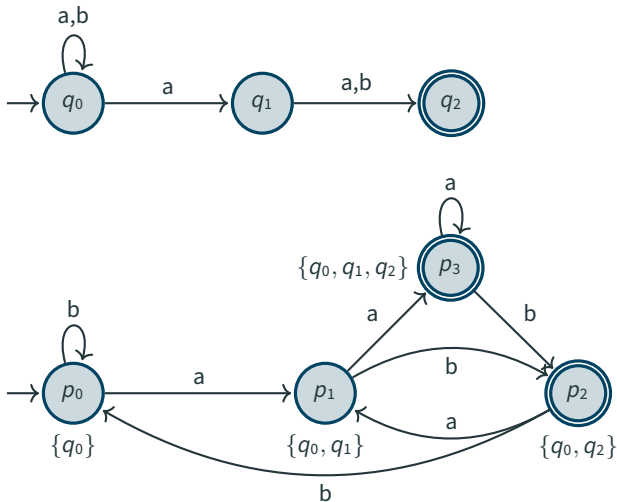
POTENZMENGENKONSTRUKTION – BEISPIEL



POTENZMENGENKONSTRUKTION – BEISPIEL



POTENZMENGENKONSTRUKTION – BEISPIEL



Theorem (Rabin und Scott – Potenzmengenkonstruktion)

Sei $A = (Q, \Sigma, \delta, Q_0, F)$ ein nichtdeterministischer endlicher Automat. Konstruiere einen deterministischen endlichen Automaten $A' = (\mathcal{P}(Q), \Sigma, \delta', Q_0, F')$ mit

- $\mathcal{P}(Q) = \{S \mid S \subseteq Q\}$, d.h. die Zustände von A' sind mit Zustandsmengen von A beschriftet*
- $\delta'(\{q_1, \dots, q_k\}, a) = \bigcup_{i=1}^k \delta(q_i, a)$, d.h. der deterministische Nachfolgezustand von $\{q_1, \dots, q_k\}$ mit Buchstabe a ist die Menge aller Zustände, die der NEA A von einem Zustand in $\{q_1, \dots, q_k\}$ mit a erreichen kann.*
- $F' = \{S \subseteq Q \mid S \cap F \neq \emptyset\}$, d.h. ein Zustand ist akzeptierend, falls er einen akzeptierenden Zustand des NEAs enthält.*

Dann akzeptieren DEA A' und NEA A die gleiche Sprache.

- Starte mit Menge der Startzustände Q_0
- Solange es noch unbearbeitete Zustände $S = \{q_1, \dots, q_k\}$ gibt
 - Für alle Buchstaben des Alphabets x
 - Für jeden Zustand $q_1, \dots, q_k$ bestimmt alle x -Nachfolger im NEA und bilde daraus die Menge S' .
 - Falls es noch keinen Zustand S' gibt, füge Zustand S' hinzu.
 - Füge Transition von S nach S' mit x hinzu.
 - Markiere S as bearbeitet.

Auf diese Weise werden nur die erreichbaren Zustände bestimmt.

Vereinigung

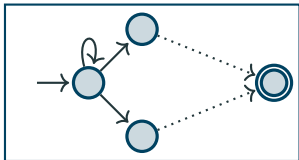
- Groß- und Kleinbuchstaben
- Oder mindestens 15 Zeichen
- Zu jeder Eigenschaft: Endlicher Automat
- Automat, der mit einem der Beiden zufrieden ist?

Satz (Abschluss gegen Vereinigung)

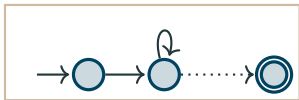
Seien A_1 und A_2 endliche Automaten, die die Sprachen $L(A_1)$ und $L(A_2)$ akzeptieren. Dann gibt es einen endlichen Automaten $A_{\cup}$, der die Vereinigung (union) akzeptiert ($L(A_{\cup}) = L(A_1) \cup L(A_2)$).

- Gegeben: Zwei Automaten
- Gesucht: Ein Automat, der akzeptiert wenn einer der Ausgangsautomaten akzeptiert.

- Schreibe beide Automaten nebeneinander

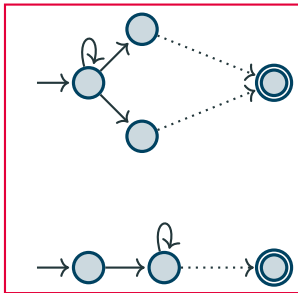


Automat A



Automat B

- Schreibe beide Automaten nebeneinander



Automat für $L(A) \cup L(B)$

Satz (Abschluss gegen Vereinigung)

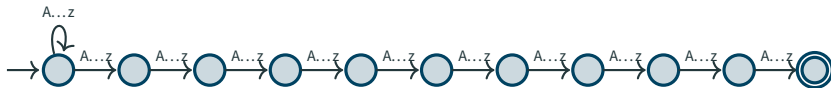
Seien $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ und $A_2 = (Q_2, \Sigma, \delta_2, Q_{02}, F_2)$ nichtdeterministische endliche Automaten. Sei $A_U = (Q_U, \Sigma, \delta_U, Q_{0U}, F_U)$ definiert durch

- $Q_U = Q_1 \cup Q_2$ (Anmerkung: disjunkte Vereinigung)
- $F_U = F_1 \cup F_2$
- $Q_{0U} = Q_{01} \cup Q_{02}$
- $\delta_U = \delta_1 \cup \delta_2$

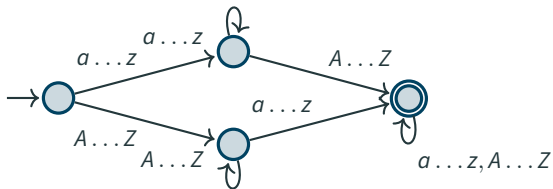
Dann gilt: $L(A_U) = L(A_1) \cup L(A_2)$.

- 1. Bedingung: “Wort hat mindestens 10 Zeichen”
- 2. Bedingung: “Groß- und Kleinbuchstaben”
- Automat für Vereinigung (“oder”)

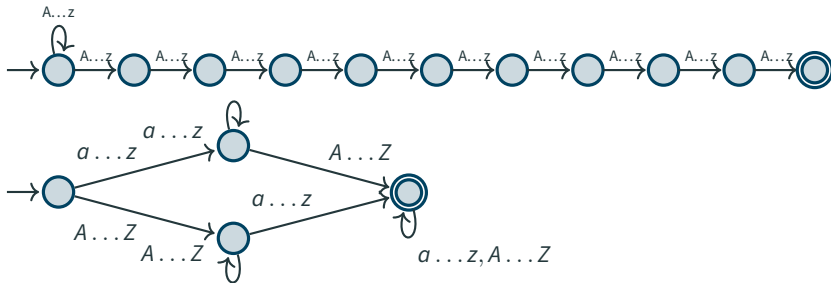
- 1. Bedingung: “Wort hat mindestens 10 Zeichen”
- 2. Bedingung: “Groß- und Kleinbuchstaben”
- Automat für Vereinigung (“oder”)



- 1. Bedingung: "Wort hat mindestens 10 Zeichen"
- 2. Bedingung: "Groß- und Kleinbuchstaben"
- Automat für Vereinigung ("oder")



- 1. Bedingung: "Wort hat mindestens 10 Zeichen"
- 2. Bedingung: "Groß- und Kleinbuchstaben"
- Automat für Vereinigung ("oder")



Komplement

- Anforderung: Keine “bösen” Sonderzeichen, z.B. kein \$
- Start: Automat zur Sprache “Wort enthält \$”
- Daraus Automat zu Sprache “Wort enthält nicht \$”?



- Anforderung: Keine “bösen” Sonderzeichen, z.B. kein \$
- Start: Automat zur Sprache “Wort enthält \$”
- Daraus Automat zu Sprache “Wort enthält nicht \$”?
- Idee: Konstruiere DEA und tausche Endzustände

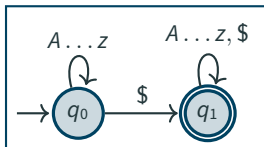
- Anforderung: Keine “bösen” Sonderzeichen, z.B. kein \$
- Start: Automat zur Sprache “Wort enthält \$”
- Daraus Automat zu Sprache “Wort enthält nicht \$”?

Satz (Abschluss unter Komplement)

Sei A_1 ein NEA, der die Sprache $L(A)$ akzeptiert. Dann gibt es einen DEA $A_D = (Q_D, \Sigma, \delta_D, q_{0D}, F_D)$ mit $L(A_1) = L(A_D)$. Der Automat $\bar{A} = (Q_D, \Sigma, \delta_D, q_{0D}, Q \setminus F)$ akzeptiert dann das Komplement (complement), d.h. $(L(\bar{A}) = \Sigma^* \setminus L(A_D))$.

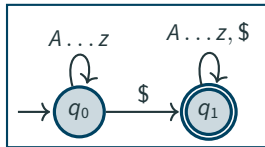
- Anforderung: Keine “bösen” Sonderzeichen, z.B. kein \$
- Start: Automat zur Sprache “Wort enthält \$”
- Daraus Automat zu Sprache “Wort enthält nicht \$”?

Automat A



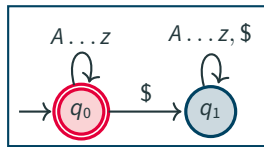
- Anforderung: Keine “bösen” Sonderzeichen, z.B. kein \$
- Start: Automat zur Sprache “Wort enthält \$”
- Daraus Automat zu Sprache “Wort enthält nicht \$”?

Automat A



Tausche akzeptierende und
nichtakzeptierende Zustände

Automat für $\Sigma^* \setminus L(A)$



Schnitt

- Groß- und Kleinbuchstaben
- Mindestens 6 Zeichen
- Endlicher Automat zu jeder einzelnen Anforderung
- Daraus Automat der Beides prüft?

Satz (Abschluss gegen Schnitt)

Seien A_1 und A_2 endliche Automaten, die die Sprachen $L(A_1)$ und $L(A_2)$ akzeptieren. Dann gibt es einen endlichen Automaten $A_\cap$, der deren Schnitt (intersection) akzeptiert ($L(A_\cap) = L(A_1) \cap L(A_2)$).

- Groß- und Kleinbuchstaben
- Mindestens 6 Zeichen
- Endlicher Automat zu jeder einzelnen Anforderung
- Daraus Automat der Beides prüft?

Satz (Abschluss gegen Schnitt)

Seien A_1 und A_2 endliche Automaten, die die Sprachen $L(A_1)$ und $L(A_2)$ akzeptieren. Dann gibt es einen endlichen Automaten $A_\cap$, der deren Schnitt (intersection) akzeptiert ($L(A_\cap) = L(A_1) \cap L(A_2)$).

Aufgabe

Überlegen Sie, wie ein solcher $A_\cap$ konstruiert werden kann.

Lösung

Erste Idee: Es gilt $L(A_1) \cap L(A_2) = \overline{\overline{L(A_1)} \cup \overline{L(A_2)}}$ und endliche Automaten sind gegen Vereinigung und Komplement abgeschlossen.

Lösung

*Zweite Idee: Simuliere A_1 und A_2 parallel. Zustandsmenge ist $Q_1 \times Q_2$, d.h. man merkt sich im Zustand sowohl den Zustand des ersten als auch des zweiten Automaten. Details:
Übungsaufgabe*

Konkatenation und Iteration

Seien L_1 und L_2 Sprachen

- Vereinigung ($L_1 \cup L_2$), Schnitt ($L_1 \cap L_2$), Komplement ($\bar{L} = \Sigma^* \setminus L$) klar
- Ideen für weitere Operationen?



Seien L_1 und L_2 Sprachen

- Vereinigung ($L_1 \cup L_2$), Schnitt ($L_1 \cap L_2$), Komplement ($\bar{L} = \Sigma^* \setminus L$) klar
- Konkatenation: $L_1 \cdot L_2 = \{w_1w_2 \mid w_1 \in L_1, w_2 \in L_2\}$

Seien L_1 und L_2 Sprachen

- Vereinigung ($L_1 \cup L_2$), Schnitt ($L_1 \cap L_2$), Komplement ($\bar{L} = \Sigma^* \setminus L$) klar
- Konkatenation: $L_1 \cdot L_2 = \{w_1w_2 \mid w_1 \in L_1, w_2 \in L_2\}$
- Potenz

$$L_1^0 = \{\varepsilon\}$$

$$L_1^{n+1} = L_1^n \cdot L_1$$

Seien L_1 und L_2 Sprachen

- Vereinigung ($L_1 \cup L_2$), Schnitt ($L_1 \cap L_2$), Komplement ($\bar{L} = \Sigma^* \setminus L$) klar
- Konkatenation: $L_1 \cdot L_2 = \{w_1 w_2 \mid w_1 \in L_1, w_2 \in L_2\}$
- Potenz

$$L_1^0 = \{\varepsilon\}$$

$$L_1^{n+1} = L_1^n \cdot L_1$$

- (Kleene-)Stern

$$L_1^* = \bigcup_{n \in \mathbb{N}} L_1^n = \{w_1 w_2 \dots w_n \mid w_1, w_2, \dots, w_n \in L_1\}$$

Seien L_1 und L_2 Sprachen

- Vereinigung ($L_1 \cup L_2$), Schnitt ($L_1 \cap L_2$), Komplement ($\bar{L} = \Sigma^* \setminus L$) klar
- Konkatenation: $L_1 \cdot L_2 = \{w_1 w_2 \mid w_1 \in L_1, w_2 \in L_2\}$
- Potenz

$$L_1^0 = \{\varepsilon\}$$

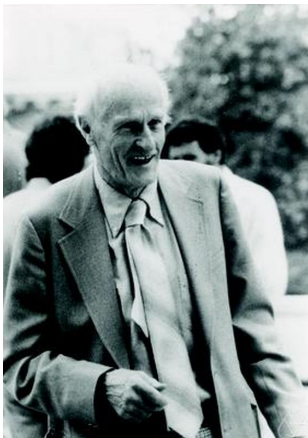
$$L_1^{n+1} = L_1^n \cdot L_1$$

- (Kleene-)Stern

$$L_1^* = \bigcup_{n \in \mathbb{N}} L_1^n = \{w_1 w_2 \dots w_n \mid w_1, w_2, \dots, w_n \in L_1\}$$

- Die endlich akzeptierbaren (regulären Sprachen) sind unter diesen (und noch mehr) Operation abgeschlossen

STEPHEN COLE KLEENE (1909 – 1994)



Stephen Cole Kleene

Photo: Mathematisches Forschungsinstitut Oberwolfach,
http://owpdb.mfo.de/detail?photo_id=2122

Konrad Jacobs, Erlangen, Copyright MFO

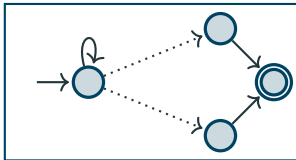
Lizenz: Creative Commons Attribution-Share Alike 2.0 Germany

- Gegeben: Zwei Automaten
- Gesucht: Ein Automat, der alle Wörter akzeptiert, deren 1. Teil vom ersten Automaten und deren zweiter Teil vom zweitem Automaten akzeptiert wird..

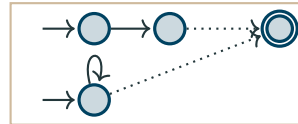


KONKATENATION - IDEE: REIHENSCHALTUNG

Automat A

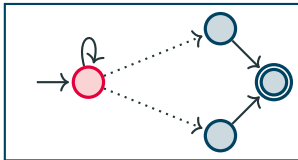


Automat B

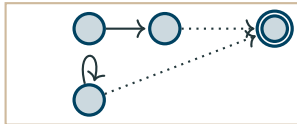


- Startzustände von erstem Automat sind Startzustände der Reihenschaltung

Automat A

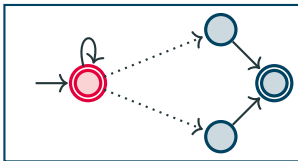


Automat B

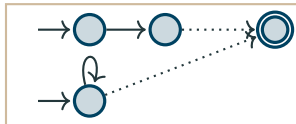


- Startzustände von erstem Automat sind Startzustände der Reihenschaltung
- Falls Startzustand vom ersten Automat auch Endzustand ist: Alle Startzustände des zweiten Automaten sind auch Startzustände

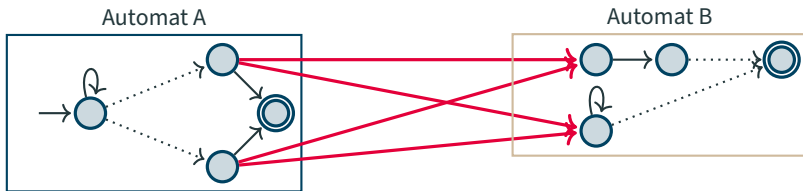
Automat A



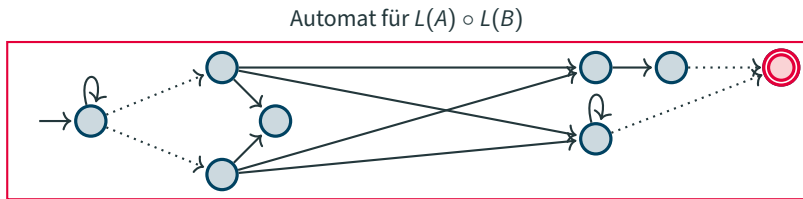
Automat B



- Startzustände von erstem Automat sind Startzustände der Reihenschaltung
- Falls Startzustand vom ersten Automat auch Endzustand ist: Alle Startzustände des zweiten Automaten sind auch Startzustände
- Für jeden Pfeil in einen Endzustand des ersten Automaten: Füge Pfeil zu jedem Startzustand des zweiten Automaten hinzu.



- Startzustände von erstem Automat sind Startzustände der Reihenschaltung
- Falls Startzustand vom ersten Automat auch Endzustand ist: Alle Startzustände des zweiten Automaten sind auch Startzustände
- Für jeden Pfeil in einen Endzustand des ersten Automaten: Füge Pfeil zu jedem Startzustand des zweiten Automaten hinzu.
- Endzustände der Reihenschaltung sind die Endzustände des zweiten Automaten



Satz (Abschluss gegen Konkatenation)

Seien $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ und $A_2 = (Q_2, \Sigma, \delta_2, Q_{02}, F_2)$ endliche Automaten. Sei $A_\bullet = (Q_\bullet, \Sigma, \delta_\bullet, Q_{0\bullet}, F_\bullet)$ definiert durch

- $Q_\bullet = Q_1 \cup Q_2$ (Anmerkung: disjunkte Vereinigung)
- $F_\bullet = F_2$
- $Q_{0\bullet} = Q_{01}$ falls $Q_{01} \cap F_1 = \emptyset$ und $Q_{0\bullet} = Q_{01} \cup Q_{02}$ sonst.
- $\delta_\bullet(q, a) = \begin{cases} \delta_1(q, a) & \text{falls } q \text{ in } A_1 \text{ liegt und } q \text{ mit } a \text{ nicht in Endzustand führt} \\ \delta_1(q, a) \cup Q_{02} & \text{falls } q \text{ in } A_1 \text{ liegt und } q \text{ mit } a \text{ in Endzustand führt} \\ \delta_2(q, a) & \text{falls } q \text{ in } A_2 \text{ liegt} \end{cases}$

Dann gilt: $L(A_\bullet) = L(A_1) \cdot L(A_2)$.

Satz (Abschluss gegen Konkatenation)

Seien $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ und $A_2 = (Q_2, \Sigma, \delta_2, Q_{02}, F_2)$ endliche Automaten. Sei $A_\bullet = (Q_\bullet, \Sigma, \delta_\bullet, Q_{0\bullet}, F_\bullet)$ definiert durch

- $Q_\bullet = Q_1 \cup Q_2$ (Anmerkung: disjunkte Vereinigung)
- $F_\bullet = F_2$
- $Q_{0\bullet} = Q_{01}$ falls $Q_{01} \cap F_1 = \emptyset$ und $Q_{0\bullet} = Q_{01} \cup Q_{02}$ sonst.
- $\delta_\bullet(q, a) = \begin{cases} \delta_1(q, a) & \text{falls } q \in Q_1 \text{ und } \delta_1(q, a) \cap F_1 = \emptyset \\ \delta_1(q, a) \cup Q_{02} & \text{falls } q \in Q_1 \text{ und } \delta_1(q, a) \cap F_1 \neq \emptyset \\ \delta_2(q, a) & \text{falls } q \in Q_2 \end{cases}$

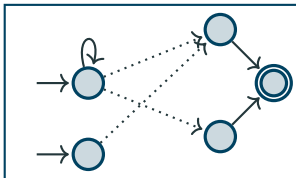
Dann gilt: $L(A_\bullet) = L(A_1) \cdot L(A_2)$.

- Gegeben: Ein Automat A
- Gesucht: Ein Automat, der alle Wörter akzeptiert, die aus Teilwörtern zusammengesetzt sind, die jeweils von A akzeptiert werden.

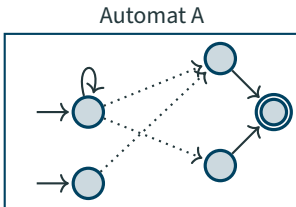


ITERATION (STERN) - IDEE: RÜCKKOPPLUNG

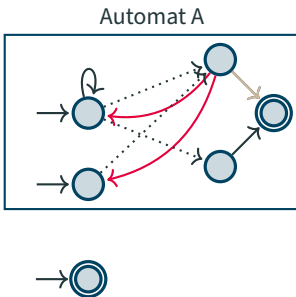
Automat A



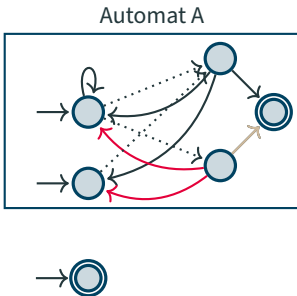
- Füge neuen Startzustand hinzu, mache ihn zum Endzustand Grund: ε muss akzeptiert werden



- Füge neuen Startzustand hinzu, mache ihn zum Endzustand Grund: ε muss akzeptiert werden
- Für jeden Pfeil in einen Endzustand: Füge Pfeil zu jedem Startzustand hinzu.

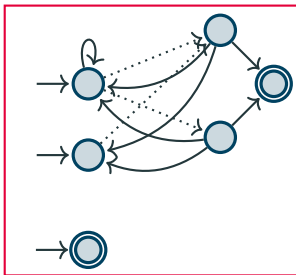


- Füge neuen Startzustand hinzu, mache ihn zum Endzustand Grund: ε muss akzeptiert werden
- Für jeden Pfeil in einen Endzustand: Füge Pfeil zu jedem Startzustand hinzu.



- Füge neuen Startzustand hinzu, mache ihn zum Endzustand Grund: ε muss akzeptiert werden
- Für jeden Pfeil in einen Endzustand: Füge Pfeil zu jedem Startzustand hinzu.

Automat für $L(A)^*$



Satz (Abschluss gegen Iteration (Stern))

Sei $A_1 = (Q_1, \Sigma, \delta_1, Q_{01}, F_1)$ ein endlicher Automat. Sei $A_* = (Q_*, \Sigma, \delta_*, Q_{0*}, F_*)$ definiert durch

- $Q_* = Q_1 \cup \{q_\epsilon\}$
- $F_* = F_1 \cup \{q_\epsilon\}$
- $Q_{0*} = Q_{01} \cup \{q_\epsilon\}$
- $\delta_*(q, a) = \begin{cases} \emptyset & \text{falls } q = q_\epsilon \\ \delta(q, a) & \text{falls } q \neq q_\epsilon \text{ und } \delta(q, a) \cap F_1 = \emptyset \\ \delta(q, a) \cup Q_{01} & \text{falls } q \neq q_\epsilon \text{ und } \delta(q, a) \cap F_1 \neq \emptyset \end{cases}$

Dann gilt: $L(A_*) = L(A_1)^*$.